Software Requirements Specification

for

Food Delivery Management System

Prepared by:

Yash Mishra(202401100200360)
Umesh Singh(202401100200327)

Shubham Singh(202401100200301)

Vidhit Hatwaliya(202401100200346)

Organization: Food Delivery Management System — Development Team

Table of Contents

Revision History

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements

- 3.1 User Interfaces
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

4. System Features

- 4.1 User Registration and Authentication
- 4.2 Restaurant and Menu Browsing
- 4.3 Order Placement and Cart Management
- 4.4 Payment Processing
- 4.5 Order Tracking and Delivery Management
- 4.6 Ratings and Reviews
- 4.7 Restaurant Menu and Order Management
- 4.8 Administrator Management and Analytics

5. Other Nonfunctional Requirements

- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes
- 5.5 Business Rules

6. Other Requirements

Appendix A: Glossary

Appendix B: Analysis Models

B.1 System Architecture Diagram

B.2 Use Case Diagram

B.3 Order Status Lifecycle (State Diagram)

B.4 Entity-Relationship Diagram

Appendix C: To Be Determined List

Revision History

Name	Date	Reason For Changes	Version
Project Team	27-Aug-2026	Initial draft of SRS	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Food Delivery Management System (FDMS), Version 1.0. FDMS is a web- and mobile-based platform that connects customers, restaurants, and delivery agents to enable online food ordering, order tracking, and delivery management. This document covers the complete system, including the customer application, the restaurant partner portal, the delivery agent application, and the administrative back-end, and is intended to serve as the authoritative reference for the design, development, and testing of Release 1.0 of the system.

1.2 Document Conventions

This document follows the IEEE 830 SRS format. Headings are numbered hierarchically (e.g., 4.2.1). Each functional requirement is uniquely tagged as REQ-<Feature>-<Number> (for example, REQ-ORD-01) so that it can be traced through design, implementation, and test artifacts. Requirement priority is expressed as High, Medium, or Low, where High indicates a requirement essential for the initial release. Terms defined in Appendix A (Glossary) appear in italics on first use. The keywords "shall" and "must" indicate a mandatory requirement; "should" indicates a recommended but non-mandatory capability.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Project Managers — for scope, planning, and milestone tracking.
- Software Developers and Architects — for functional and interface requirements guiding design and implementation.
- Quality Assurance / Test Engineers — for deriving test cases and acceptance criteria.
- UI/UX Designers — for user interface and interaction requirements.
- Business Stakeholders and Restaurant Partners — for validating that the specified system meets business needs.

Readers new to the project should first read Sections 1 and 2 for context and scope, then proceed to Section 3 (interfaces), Section 4 (feature-level functional requirements), and Section 5 (non-functional requirements). Developers implementing a specific feature may go directly to the relevant subsection of Section 4.

1.4 Product Scope

The Food Delivery Management System is a multi-actor platform designed to digitize and streamline the food ordering and delivery process. It allows customers to browse nearby restaurants and menus, place and pay for orders, and track deliveries in real time. Restaurant partners can manage their menus, accept or reject incoming orders, and monitor sales performance. Delivery agents can accept delivery assignments, navigate to pickup and drop-off locations, and update delivery status. Administrators oversee the platform through a dashboard providing user, restaurant, and order management along with analytics and reporting.

The objective of FDMS is to reduce order-processing time, minimize manual errors in order handling, provide transparency to customers through real-time tracking, and give restaurant partners tools to grow their business. This release (Version 1.0) targets a single-city deployment with support for future multi-city scaling.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Food Delivery Management System — Vision and Scope Document, v1.0 (internal).
- Food Delivery Management System — UI/UX Style Guide (internal, to be delivered).
- Payment Gateway API Integration Guide (third-party provider documentation).
- Google Maps Platform / GPS Navigation API Documentation.

2. Overall Description

2.1 Product Perspective

The Food Delivery Management System is a new, self-contained, cloud-hosted product composed of four client-facing applications (Customer App, Restaurant Portal, Delivery Agent App, and Admin Dashboard) that communicate with a shared set of back-end services through a common API Gateway. The back-end integrates with third-party services for payment processing, maps/GPS navigation, and SMS/push notifications. Figure 2.1 illustrates the major components of the system, their interconnections, and the external interfaces.

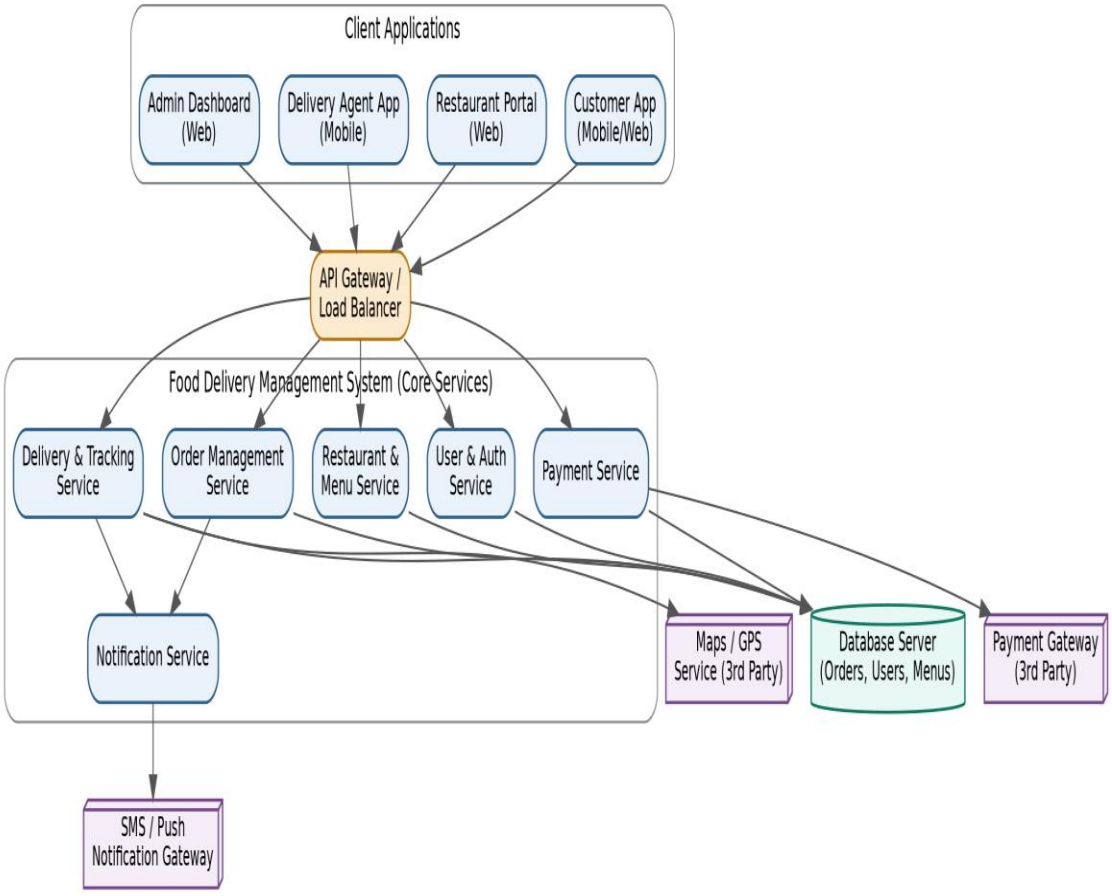

Figure 2.1 — Food Delivery Management System: High-Level System Architecture

2.2 Product Functions

The major functions the system must provide, grouped by actor, are summarized below and detailed in Section 4. Figure 2.2 shows these functions and the actors that use them in a use-case view.

- Customer: account registration/login, restaurant and menu browsing, cart management, order placement, online payment, real-time order tracking, ratings and reviews.
- Restaurant: menu management, order acceptance/rejection, order status updates, sales/performance visibility.

- Delivery Agent: delivery assignment acceptance, navigation, pickup/drop-off status updates, live location sharing.
- Administrator: user and restaurant account management, dispute handling, platform-wide analytics and reporting.

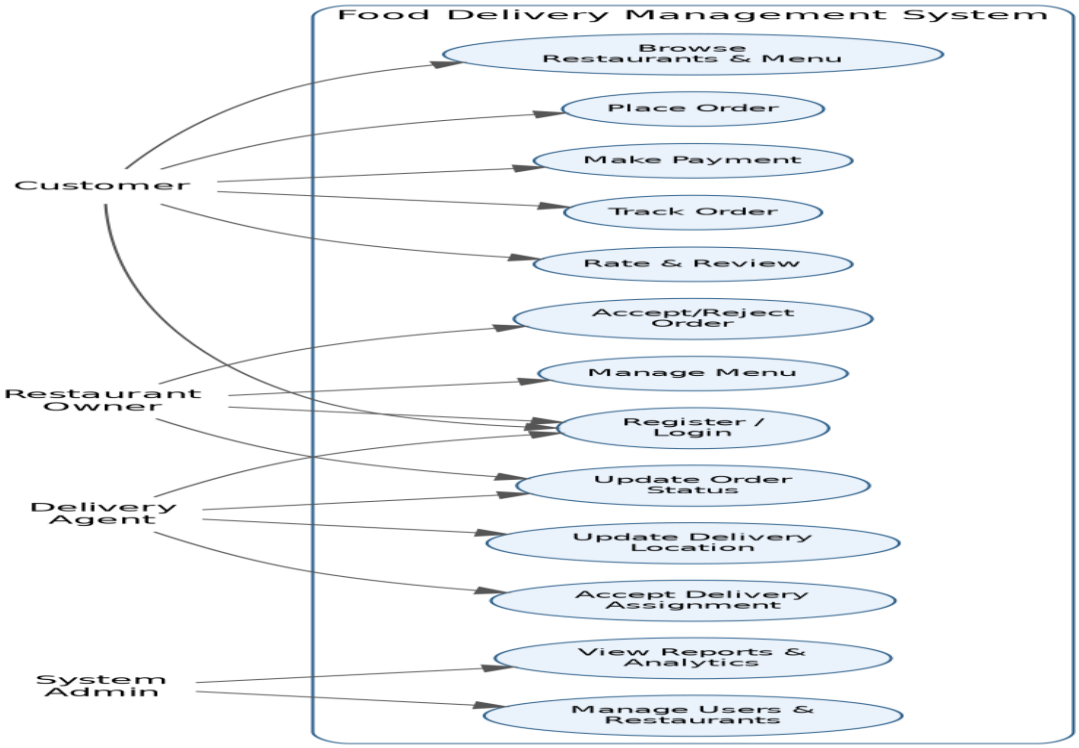

Figure 2.2 — Use Case Diagram of the Food Delivery Management System

2.3 User Classes and Characteristics

User Class	Characteristics	Frequency of Use	Importance
Customer	General public; varying technical expertise; uses mobile app primarily	Multiple times/week	High
Restaurant Staff	Restaurant owners/managers; moderate technical expertise; uses web portal/tablet	Continuous during business hours	High
Delivery Agent	Field workers; basic technical expertise; uses mobile app with GPS	Continuous during shift	High
Administrator	Platform operations staff; high technical expertise; elevated privileges	Daily	Medium

Customers and Restaurant Staff are the most critical user classes for adoption and revenue; the system must prioritize ease of use for these groups. Administrators require fewer but more powerful capabilities and are assumed to be trained platform staff.

2.4 Operating Environment

- Customer & Restaurant mobile apps: Android 9.0+ and iOS 14.0+.
- Web portals (Restaurant Portal, Admin Dashboard): latest two major versions of Chrome, Firefox, Edge, and Safari.
- Server-side: containerized microservices deployed on a cloud platform (e.g., AWS/Azure/GCP), Linux-based OS.
- Database: relational database management system (e.g., PostgreSQL/MySQL) with support for horizontal read scaling.
- Network: system requires stable internet connectivity for all client applications; delivery agent app must tolerate intermittent connectivity.

2.5 Design and Implementation Constraints

- The system must comply with applicable data-protection and e-commerce regulations in the deployment region.
- Payment processing must use a PCI-DSS compliant third-party payment gateway; the system must not store raw card data.
- The system must be built using a microservices architecture to allow independent scaling of order, delivery, and payment services.
- Real-time location tracking requires integration with a licensed maps/GPS API with rate and cost limits.
- Mobile applications must be developed for cross-platform delivery (Android and iOS) within the project timeline.
- All inter-service and client-server communication must use HTTPS/TLS.

2.6 User Documentation

- In-app onboarding tutorials and tooltips for Customer, Restaurant, and Delivery Agent applications.
- Online Help Center / FAQ accessible from all client applications.
- Restaurant Partner Onboarding Guide (PDF and web page).
- Administrator User Manual describing dashboard operations and reporting.

2.7 Assumptions and Dependencies

- It is assumed that customers, restaurants, and delivery agents have access to smartphones or computers with internet connectivity.
- The system depends on a third-party payment gateway for transaction processing; gateway downtime will affect payment-related functions.
- The system depends on a third-party maps/GPS provider for geocoding, distance calculation, and live tracking.
- It is assumed that restaurant partners will keep their menu and availability data reasonably up to date.

- SMS/push notification delivery depends on the availability of the chosen third-party gateway provider.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide four distinct user interfaces, each tailored to its user class:

- Customer App: home screen with restaurant listings and search/filter; restaurant menu screen; cart and checkout screen; live order-tracking screen with map view; order history and ratings screen.
- Restaurant Portal: dashboard with incoming order queue; menu management screen (add/edit/remove items, set availability); sales and order-history reports.
- Delivery Agent App: available-deliveries screen; active-delivery screen with turn-by-turn navigation; status-update controls (Picked Up, Out for Delivery, Delivered).
- Admin Dashboard: user and restaurant management screens; order monitoring; analytics and reporting screens; dispute/complaint management.

All interfaces shall follow a common visual style guide, provide standard navigation and help controls, and display clear error messages for invalid inputs (e.g., failed payment, out-of-stock item, unreachable delivery agent). Screen mockups will be documented in a separate UI Specification.

3.2 Hardware Interfaces

- Customer and Delivery Agent mobile apps shall interface with the device's GPS/location hardware to determine current location.
- Mobile apps shall interface with the device's push-notification hardware/OS services to receive real-time alerts.
- Restaurant Portal may optionally interface with a receipt/order printer via the browser's print service.
- No specialized or proprietary hardware is required beyond standard smartphones, tablets, and computers with camera/GPS capability.

3.3 Software Interfaces

Interface	Purpose	Data Exchanged
Payment Gateway API	Process online payments and refunds	Order amount, payment token, transaction status
Maps/GPS API	Geocoding, distance/ETA calculation, live tracking	Coordinates, addresses, route, ETA
SMS/Push Gateway	Deliver order and delivery status notifications	Recipient contact, message content, delivery receipt
Relational Database	Persistent storage for users, restaurants, orders, payments	Structured records via ORM/SQL

All internal microservices communicate via RESTful JSON APIs behind the API Gateway (see Figure 2.1). API contracts will be documented separately using OpenAPI specifications.

3.4 Communications Interfaces

- All client-server communication shall use HTTPS (TLS 1.2 or higher).
- Real-time order and delivery status updates shall be delivered via WebSocket or a push-notification mechanism to minimize polling.
- The system shall communicate with the payment gateway over encrypted REST API calls.
- SMS notifications shall be sent via a secure API gateway conforming to the provider's authentication requirements.
- Data transfer between mobile apps and the server shall be optimized for low-bandwidth conditions common in mobile networks.

4. System Features

This section organizes functional requirements by system feature, reflecting the major services the product provides across its four user classes.

4.1 User Registration and Authentication

4.1.1 Description and Priority

Allows customers, restaurant staff, and delivery agents to create accounts, log in securely, and manage their profiles. Priority: High.

4.1.2 Stimulus/Response Sequences

- User submits registration form → system validates input, creates account, sends verification code.
- User enters verification code → system activates account and logs the user in.
- User submits login credentials → system authenticates and issues a session token.
- User requests password reset → system sends a reset link/OTP to the registered email or phone.

4.1.3 Functional Requirements

ID	Description	Priority
REQ-AUTH-01	The system shall allow a new user to register using email or phone number, and a password.	High
REQ-AUTH-02	The system shall verify a new account via a one-time code sent by SMS or email before activation.	High
REQ-AUTH-03	The system shall allow registered users to log in using their credentials and issue a secure session token.	High
REQ-AUTH-04	The system shall allow a user to reset a forgotten password via email or SMS verification.	Medium
REQ-AUTH-05	The system shall lock an account for 15 minutes after 5 consecutive failed login attempts.	Medium
REQ-AUTH-06	The system shall allow users to update their profile information (name, address, contact number).	Medium

4.2 Restaurant and Menu Browsing

4.2.1 Description and Priority

Allows customers to discover restaurants and view menus based on location, cuisine, and search criteria. Priority: High.

4.2.2 Stimulus/Response Sequences

- Customer opens the app / allows location access → system displays nearby restaurants sorted by distance or rating.

- Customer searches by restaurant name, cuisine, or dish → system returns matching results.
- Customer selects a restaurant → system displays the full menu with categories, prices, and availability.

4.2.3 Functional Requirements

ID	Description	Priority
REQ-BRW-01	The system shall display a list of restaurants within a configurable delivery radius of the customer's location.	High
REQ-BRW-02	The system shall allow customers to filter restaurants by cuisine type, rating, and estimated delivery time.	Medium
REQ-BRW-03	The system shall allow customers to search menus and restaurants by keyword.	Medium
REQ-BRW-04	The system shall display menu item details including price, description, and current availability.	High

4.3 Order Placement and Cart Management

4.3.1 Description and Priority

Allows customers to add items to a cart, customize items, and place an order with the selected restaurant. Priority: High.

4.3.2 Stimulus/Response Sequences

- Customer adds item(s) to cart → system updates the cart total in real time.
- Customer proceeds to checkout → system displays order summary, delivery address, and payment options.
- Customer confirms order → system validates item availability, creates the order, and forwards it to the restaurant.

4.3.3 Functional Requirements

ID	Description	Priority
REQ-ORD-01	The system shall allow a customer to add, remove, or modify item quantities in the cart before checkout.	High
REQ-ORD-02	The system shall validate item availability at the time of order placement and reject unavailable items with a clear message.	High
REQ-ORD-03	The system shall allow the customer to select or add a delivery address at checkout.	High
REQ-ORD-04	The system shall calculate order total including item cost, taxes, delivery fee, and applicable discounts.	High
REQ-ORD-05	The system shall generate a unique order ID and forward the order to the selected restaurant upon confirmation.	High
REQ-ORD-06	The system shall allow a customer to cancel an order only while it is in the "Placed" or "Confirmed" state.	Medium

4.4 Payment Processing

4.4.1 Description and Priority

Enables secure online payment for orders through integrated payment methods, as well as cash-on-delivery. Priority: High.

4.4.2 Stimulus/Response Sequences

- Customer selects a payment method at checkout → system redirects to/invokes the payment gateway.
- Payment gateway returns a success/failure response → system updates order status accordingly and notifies the customer.
- Customer requests a refund for a cancelled order → system initiates the refund via the payment gateway.

4.4.3 Functional Requirements

ID	Description	Priority
REQ-PAY-01	The system shall support payment via credit/debit card, digital wallet, UPI, and cash-on-delivery.	High
REQ-PAY-02	The system shall process card and digital payments through a PCI-DSS compliant third-party payment gateway.	High
REQ-PAY-03	The system shall not store raw payment card details on its own servers.	High
REQ-PAY-04	The system shall confirm the order only after successful payment authorization (except for cash-on-delivery).	High
REQ-PAY-05	The system shall automatically initiate a refund to the original payment method for a cancelled paid order.	Medium

4.5 Order Tracking and Delivery Management

4.5.1 Description and Priority

Manages the lifecycle of an order from restaurant confirmation through delivery, including delivery-agent assignment and real-time tracking. Priority: High. Figure 4.1 shows the order status lifecycle used by this feature.

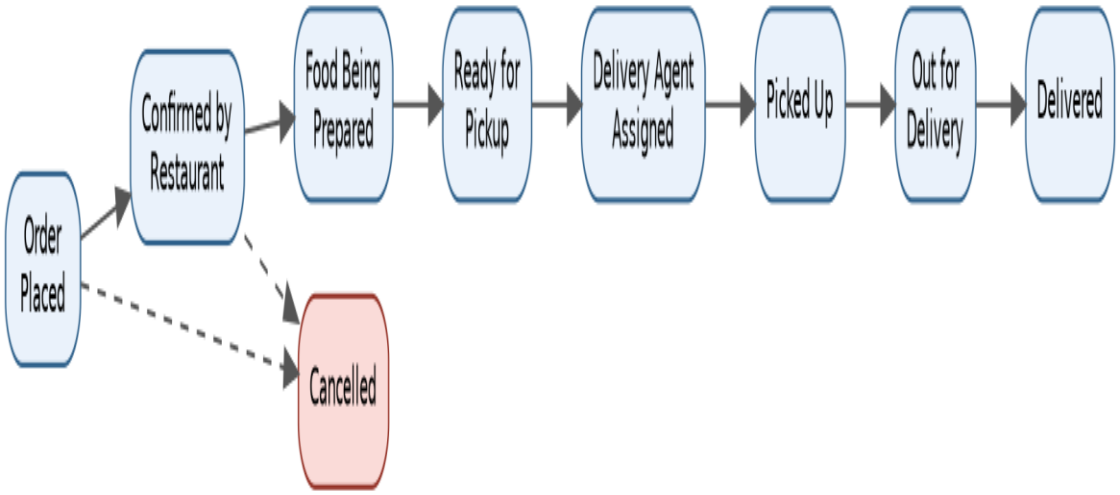

Figure 4.1 — Order Status Lifecycle

4.5.2 Stimulus/Response Sequences

- Restaurant confirms an order → system marks the order "Confirmed" and notifies the customer.
- Restaurant marks food "Ready for Pickup" → system assigns the nearest available delivery agent.
- Delivery agent accepts assignment → system notifies the customer and shares agent details and live location.
- Delivery agent updates status to "Picked Up," "Out for Delivery," or "Delivered" → system updates order status and notifies the customer at each step.

4.5.3 Functional Requirements

ID	Description	Priority
REQ-DLV-01	The system shall allow restaurant staff to accept or reject an incoming order within a configurable time window.	High
REQ-DLV-02	The system shall automatically assign the nearest available delivery agent once an order is marked ready for pickup.	High
REQ-DLV-03	The system shall allow a delivery agent to accept or decline an assignment.	High
REQ-DLV-04	The system shall display the delivery agent's live location to the customer on a map during active delivery.	High
REQ-DLV-05	The system shall update and display the current order status (Placed, Confirmed, Preparing, Ready for Pickup, Picked Up, Out for Delivery, Delivered, Cancelled) in real time.	High
REQ-DLV-06	The system shall notify the customer via push notification/SMS at each major status change.	Medium

4.6 Ratings and Reviews

4.6.1 Description and Priority

Allows customers to rate and review restaurants and delivery experiences after order completion. Priority: Medium.

4.6.2 Stimulus/Response Sequences

- Order status changes to "Delivered" → system prompts the customer to rate the restaurant and delivery experience.
- Customer submits a rating and optional comment → system stores the review and updates the restaurant's average rating.

4.6.3 Functional Requirements

ID	Description	Priority
REQ-RAT-01	The system shall prompt the customer to rate the order (1–5 stars) after delivery is marked complete.	Medium
REQ-RAT-02	The system shall allow customers to submit an optional written review along with the rating.	Low
REQ-RAT-03	The system shall recompute and display a restaurant's average rating whenever a new review is submitted.	Medium

4.7 Restaurant Menu and Order Management

4.7.1 Description and Priority

Provides restaurant staff with tools to manage menu content, availability, and incoming orders. Priority: High.

4.7.2 Stimulus/Response Sequences

- Restaurant staff adds/edits a menu item → system updates the live menu visible to customers.
- Restaurant staff marks an item unavailable → system hides or disables ordering for that item immediately.
- New order arrives → system displays it in the restaurant's order queue with an audible/visual alert.

4.7.3 Functional Requirements

ID	Description	Priority
REQ-RST-01	The system shall allow restaurant staff to add, edit, and remove menu items, including name, price, description, and image.	High
REQ-RST-02	The system shall allow restaurant staff to toggle the availability of individual menu items in real time.	High
REQ-RST-03	The system shall display incoming orders to restaurant staff in a real-time queue with order details.	High
REQ-RST-04	The system shall allow restaurant staff to view historical orders and daily/weekly sales summaries.	Medium

4.8 Administrator Management and Analytics

4.8.1 Description and Priority

Provides administrators with tools to manage platform users, restaurants, and disputes, and to view system-wide analytics. Priority: Medium.

4.8.2 Stimulus/Response Sequences

- Administrator searches for a user/restaurant account → system displays account details and status.
- Administrator suspends or reinstates an account → system updates account status and access rights immediately.
- Administrator opens the analytics dashboard → system displays order volume, revenue, and delivery-performance metrics.

4.8.3 Functional Requirements

ID	Description	Priority
REQ-ADM-01	The system shall allow an administrator to view, suspend, or reinstate customer, restaurant, and delivery-agent accounts.	Medium
REQ-ADM-02	The system shall allow an administrator to approve or reject new restaurant partner registrations.	Medium
REQ-ADM-03	The system shall provide a dashboard showing order volume, revenue, and average delivery time over a selectable date range.	Medium
REQ-ADM-04	The system shall allow an administrator to view and respond to customer complaints/disputes linked to specific orders.	Low

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Description	Priority
REQ-PERF-01	The system shall load the restaurant listing screen within 2 seconds under normal network conditions (95th percentile).	High
REQ-PERF-02	The system shall process an order placement request (validation + creation) within 3 seconds.	High
REQ-PERF-03	The system shall support at least 10,000 concurrent active users during peak meal hours without service degradation.	Medium
REQ-PERF-04	Live delivery-location updates shall refresh on the customer's map at least once every 10 seconds.	Medium

5.2 Safety Requirements

- The system shall never expose a delivery agent's exact home address or personal contact details to customers; only masked/relayed communication shall be used.
- The system shall validate delivery addresses to reduce the risk of misdirected deliveries.
- The system shall provide an in-app emergency/support contact option for delivery agents.

5.3 Security Requirements

- The system shall encrypt all data in transit using TLS 1.2 or higher and sensitive data at rest (e.g., passwords hashed with a strong algorithm such as bcrypt).
- The system shall enforce role-based access control distinguishing Customer, Restaurant, Delivery Agent, and Administrator privileges.
- The system shall comply with PCI-DSS requirements for any component that handles payment card data, primarily by delegating card handling to a certified payment gateway.
- The system shall log all administrative actions for audit purposes.
- The system shall comply with applicable data-privacy regulations governing the collection and use of personal and location data.

5.4 Software Quality Attributes

- Availability: The system shall maintain 99.5% uptime for core ordering services, measured monthly.
- Usability: A first-time customer shall be able to complete a basic order (browse → cart → checkout) without external assistance.
- Scalability: The architecture shall support horizontal scaling of order and delivery services to accommodate demand growth and future multi-city expansion.
- Maintainability: Services shall be independently deployable to allow updates to one feature without requiring redeployment of the entire system.

- Reliability: Order and payment data shall not be lost even if a service instance fails mid-transaction (atomic order creation).
- Portability: Mobile applications shall run on both Android and iOS from a shared codebase where feasible.

5.5 Business Rules

- Only verified restaurant accounts approved by an administrator may appear in customer search results.
- An order may be cancelled by the customer only before the restaurant marks it "Preparing."
- A delivery agent may hold only one active delivery assignment at a time.
- Refunds for cancelled paid orders shall be processed automatically within the payment gateway's standard refund window.
- Restaurant partners are charged a commission (percentage of order value, as defined in the partner agreement) on each completed order.

6. Other Requirements

- Database Requirements: the system shall retain order and transaction records for a minimum of 3 years for auditing and dispute-resolution purposes.
- Internationalization: Version 1.0 targets a single-locale deployment; the architecture should not preclude adding multi-language and multi-currency support in a future release.
- Legal Requirements: the platform shall include Terms of Service and a Privacy Policy accepted by all users during registration.
- Reuse Objectives: authentication and notification services should be designed as reusable modules for potential future products.

Appendix A: Glossary

Term	Definition
FDMS	Food Delivery Management System — the product specified in this document.
Customer	An end user who browses restaurants and places food orders.
Delivery Agent	A field worker who picks up prepared food from a restaurant and delivers it to the customer.
Order	A customer's request to purchase one or more menu items from a specific restaurant.
ETA	Estimated Time of Arrival — the predicted time for order delivery.
API Gateway	A server that acts as a single entry point, routing client requests to the appropriate back-end microservice.
PCI-DSS	Payment Card Industry Data Security Standard — a security standard for organizations handling card payments.
OTP	One-Time Password — a single-use code sent for verification or authentication.
TBD	To Be Determined — placeholder for information not yet finalized.

Appendix B: Analysis Models

This appendix contains supporting analysis models referenced throughout this SRS.

B.1 System Architecture Diagram

See Figure 2.1 in Section 2.1 for the high-level component architecture.

B.2 Use Case Diagram

See Figure 2.2 in Section 2.2 for the actor/use-case view of system functions.

B.3 Order Status Lifecycle (State Diagram)

See Figure 4.1 in Section 4.5 for the state transitions an order passes through.

B.4 Entity-Relationship Diagram

Figure B.1 shows the core data entities of the Food Delivery Management System and their relationships, supporting the functional requirements in Section 4.

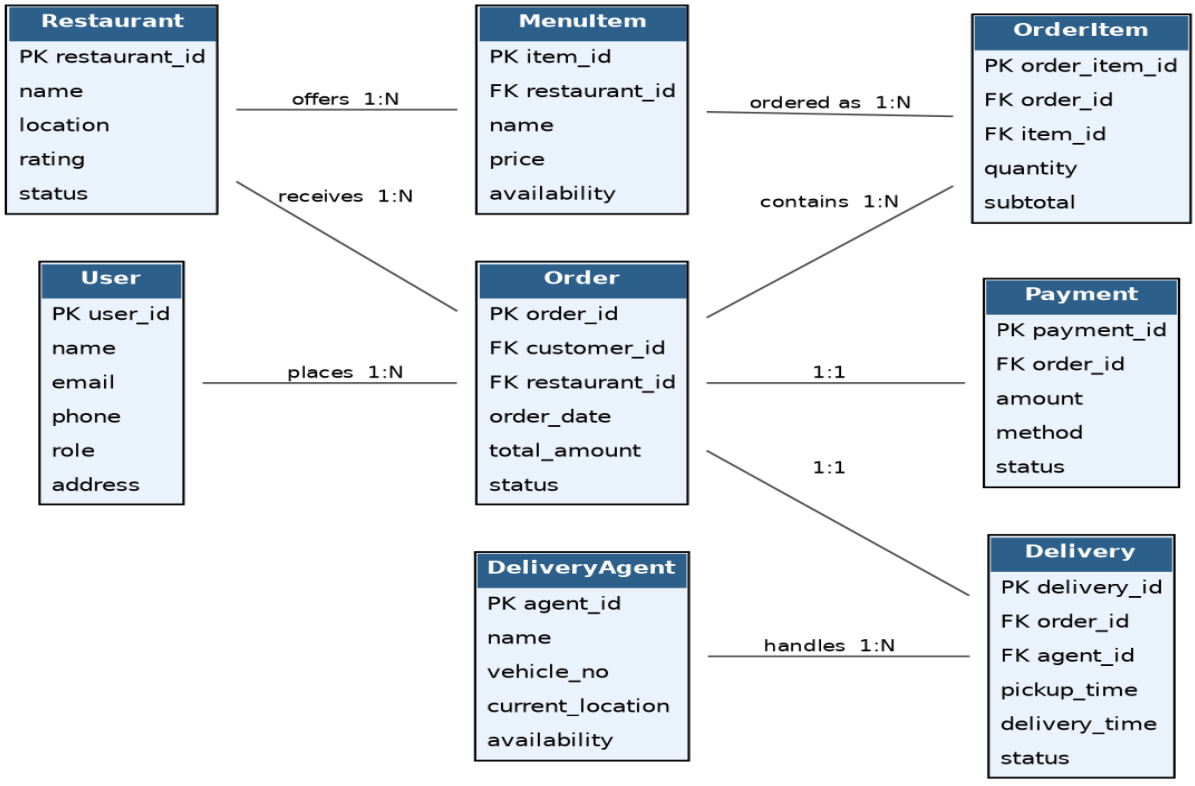

Figure B.1 — Entity-Relationship Diagram of the Food Delivery Management System

Appendix C: To Be Determined List

TBD #	Description	Section
TBD-1	Final list of supported payment methods and payment gateway vendor selection.	4.4
TBD-2	Exact commission percentage charged to restaurant partners per order.	5.5
TBD-3	Target cities/regions and localization requirements for phase 2 rollout.	2.4 / 6
TBD-4	Specific SLA terms with the maps/GPS and SMS/push-notification providers.	2.7 / 3.3